Match the Git command to the GUI action

A live matching activity for the Git for Science workshop

How this works

In this workshop we drive Git through a graphical interface: buttons in the RStudio Git pane and on GitHub. Every click runs a Git command underneath. Clicking the push button *is* `git push`. Using the interface is not “cheating”; it runs the same commands a terminal user types, and it is the bridge to the agentic-coding follow-up where an agent drives Git by naming those same commands.

The activity: draw a line from each Git command on the left to its matching GUI action on the right. Work **3 minutes on your own**, then **3 minutes with your neighbour** to compare and agree. Then we review as a group. One item on the left is a GitHub feature, not a Git command; matching it is part of the point.

Learner sheet

The dots mark where to start and end each line.

1. `git clone <url>` •
2. `git add <file>` / stage •
3. `git commit -m "..."` •
4. `git push` •
5. `git pull` •
6. `git status` •
7. `git log` •
8. `git branch <name>` / checkout •
9. `git revert <commit>` •
10. `git add` + `git commit` (on GitHub) •
11. open a pull request (GitHub feature, not a Git command) •

- • A. Click the **down-arrow** (pull) button in the RStudio Git pane
- • B. Write a commit message, then click **Commit**
- • C. On GitHub, click **Add file > Create new file** or **Upload files**, then **Commit changes**
- • D. Copy the HTTPS URL, then **File > New Project > Version Control > Git** in RStudio
- • E. Click the **up-arrow** (push) button in the RStudio Git pane
- • F. Tick the **checkbox** next to a file in the RStudio Git pane
- • G. On GitHub, open the **Pull requests** tab and click **New pull request**
- • H. Read the file list and status flags shown in the RStudio Git pane
- • I. Click the **branch dropdown** in the Git pane and choose **New Branch**
- • J. Right-click a commit in **History** and choose **Revert**, or use the Git pane menu
- • K. Click **History** (the clock icon) in the RStudio Git pane